MORE THAN JUST A PROMPT.

Multi-Agent Orchestration with the Microsoft Agent Framework

MEDIALESSON
Digital Excellence
Delivered.

SPEAKER

Sebastian Jensen

Senior Software Engineer & Team Lead
Medialesson

jensen@medialesson.de

MICROSOFT AGENT FRAMEWORK

Introduction

What is the Microsoft Agent Framework?

- Microsoft Agent Framework is Microsoft's **software development kit** for building **AI agents** and **multi-agent workflows**.

- The framework is centered around two core concepts:
  - agents for open-ended, conversational, and tool-using AI behavior
  - workflows for explicit, graph-based orchestration across multiple agents and functions

Differences between Chat (Completion) and Agent

Using Chat Completion

Program.cs

```
var response =
  await chatClient.GetResponseAsync("Tell me a joke about the IT.");

Console.WriteLine(response.Text);
```

AGENT

Using an Agent

Program.cs

```
var joker =
  chatClient.CreateAIAgent(instructions: "You are good at telling jokes.");

var response =
  await joker.RunAsync("Tell me a funny IT joke.");

Console.WriteLine(response.Text);
```

Key Differences

CHAT COMPLETION	AGENT
Stateless – each call is independent	Maintains conversation thread using a session
You manage the message history	Agent manages context internally
You handle tool calling manually	Agent handles tool invocation automatically
You write the orchestration logic	Agent reasons and decides next steps

Useful NuGet Packages #1

- **Microsoft.Agents.AI**
  - Provides the core agent abstractions for creating, running, and interacting with AI agents in .NET.

- **Microsoft.Agents.AI.Workflows**
  - Adds orchestration primitives for building structured multi-agent workflows such as sequential, concurrent, handoff, and group-chat patterns.

NUGET PACKAGES

Useful NuGet Packages #2

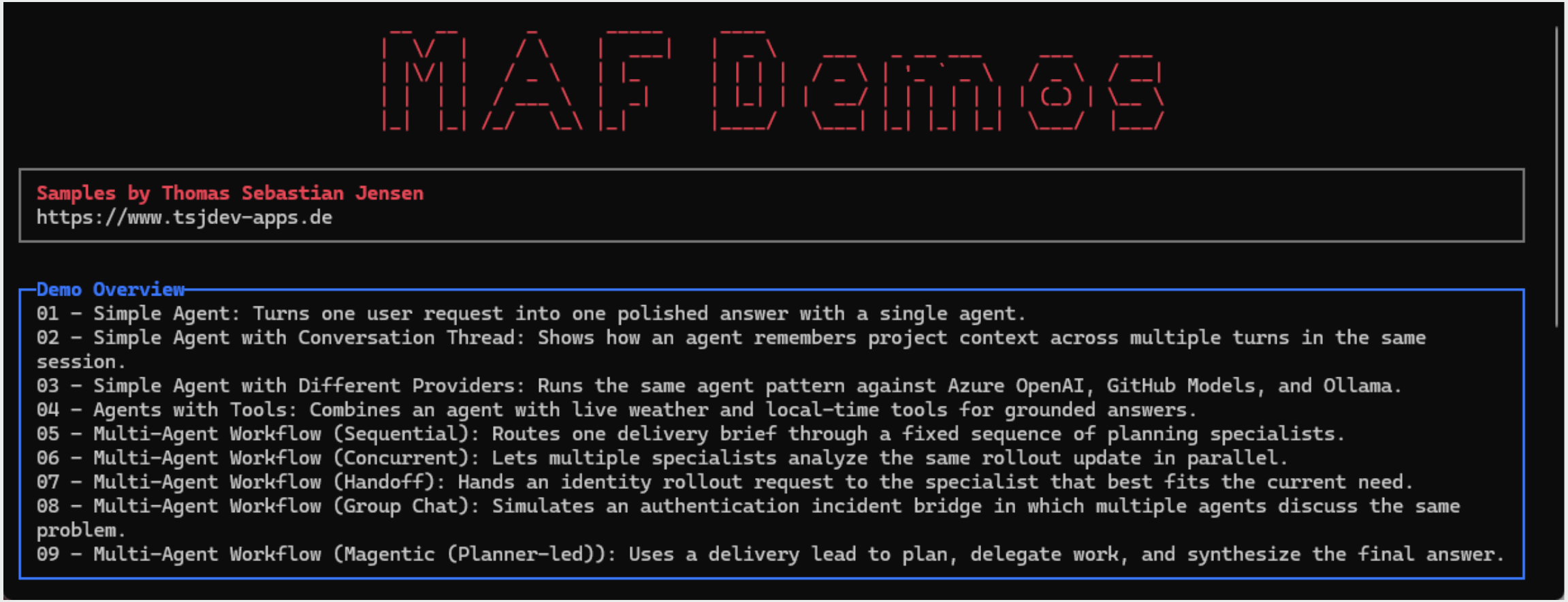
- **Microsoft.Agents.AI.OpenAI**
  - Connects Microsoft Agent Framework agents to OpenAI and Azure OpenAI models so they can be used through the shared agent APIs.

- **OllamaSharp**
  - Enables integration with Ollama-hosted local models, which is useful for provider comparisons and local demo scenarios.

Let's Talk & Code!

DEMO APPLICATION

Nine Use-Cases – One Console App



Simple Agent with Conversation Thread

Program.cs

```
AgentSession session =
    await agent.CreateSessionAsync();

await agent.RunStreamingAsync(
    userMessage, session);
```

You can easily use different Providers




AGENT TOOLS

Create a Weather Agent Tool #1

Program.cs

```
[Description("Get the current weather for a given location.")]
private static async Task<object> GetWeatherAsync(
    [Description("The location to get the weather for.")]
    string location)
{
    // logic to get the weather information
    return $"In {location} there are 15 °C.";
}
```

AGENT TOOLS

Create a Weather Agent Tool #2

Program.cs

```
AIFunction weatherTool =
  AIFunctionFactory.Create(GetWeatherAsync);

AIAgent weatherAssistant = chatClient.AsAIAgent(
    name: "weather-assistant",
    instructions: "You are a helpful assistant that can "
                + "check the weather for a city. Use the available "
                + "tool whenever the user asks for weather information.",
    tools: [weatherTool]);
```

AGENT ORCHESTRATION

Let's Talk & Code!

Five Orchestration Patterns

PATTERN	DESCRIPTION	USE CASE
Sequential	Agents process in order, passing results forward	Pipelines, step-by-step workflows
Concurrent	Agents work in parallel on the same input	Parallel analysis, ensemble decisions
Handoff	Agents dynamically pass control based on context	Expert escalation, dynamic routing
Group Chat	Agents collaborate in a shared conversation	Iterative refinement, collaborative problem-solving
Magentic	Lead agent directs other agents (planner-based)	Complex, generalist collaboration

SEQUENTIAL WORKFLOW

Output from one Agent becomes input to the next

- **Use case**
  - A delivery brief for a new feature is created, refined, and turned into a presentation-ready outcome step by step.
- **Agents**
  - Scope Analyst, Solution Drafter, Architecture Reviewer, Executive Editor
- **Why it fits**
  - The process is structured, each agent has a fixed responsibility, and every stage builds on the previous one.

SEQUENTIAL WORKFLOW

Output from one Agent becomes input to the next

Program.cs

```
Workflow pipeline =
    AgentWorkflowBuilder.BuildSequential(
        scopeAnalyst, solutionDrafter,
        architectureReviewer, executiveEditor);
```

CONCURRENT WORKFLOW

Agents can work independently on the same input

- **Use case**
  - A rollout update is evaluated at the same time from different perspectives, such as leadership, risk, and metrics.
- **Agents**
  - Leadership Brief, Risk Lens, Signal Spotter
- **Why it fits**
  - All agents work on the same input in parallel and produce fast, complementary insights.

CONCURRENT WORKFLOW

Agents can work independently on the same input

Program.cs

```
Workflow pipeline =
    AgentWorkflowBuilder.BuildConcurrent(
        LeadershipBrief, RiskLens,
        SignalSpotter);
```

HANDOFF WORKFLOW

Dynamic routing where Agents pass control

- **Use case**
  - A rollout recommendation for an identity feature is dynamically passed between security, support, and rollout specialists.
- **Agents**
  - Identity Lead, Security Lead, Support Lead, Rollout Lead
- **Why it fits**
  - The next responsible agent depends on the current focus, so a flexible handoff is more realistic than a fixed pipeline.

HANDOFF WORKFLOW

Dynamic routing where Agents pass control

Program.cs

```
Workflow handoffWorkflow = AgentWorkflowBuilder
    .CreateHandoffBuilderWith(identityLead)
    .WithHandoff(identityLead, securityLead, "Hand off when ...")
    .WithHandoff(identityLead, supportLead, "Hand off when ...")
    .WithHandoff(identityLead, rolloutLead, "Hand off when ...")
    .Build();
```

GROUP CHAT

Discussion between Agents

- **Use case**
  - Multiple agents discuss an authentication incident together, similar to a real incident bridge call.
- **Agents**
  - Bridge Lead, Identity Engineer, Reliability Engineer
- **Why it fits**
  - This pattern shows how agents can react to each other directly and build a shared understanding of the situation.

GROUP CHAT

Discussion between Agents

Program.cs

```
Workflow groupChatWorkflow = AgentWorkflowBuilder
    .CreateGroupChatBuilderWith(static agents =>
    {
        RoundRobinGroupChatManager manager = new(agents
        {
            MaximumIterationCount = 8
        };
        return manager;
    })
    .AddParticipants([bridgeLead, identityEngineer, reliabilityEngineer])
    .Build();
```

MULTI-MODEL ORCHESTRATION

Different Agents can use different AI models

- **Cost optimization**
  - Use expensive models only where needed
- **Privacy**
  - Keep sensitive data on local models
- **Capabilities**
  - Match model strengths to tasks

SUMMARY

Quick Summary

SUMMARY

Quick Summary #1

- One **consistent programming model** for single-agent, tool-enabled, and multi-agent applications
- Work across **different model providers** such as Microsoft Foundry, Azure OpenAI, GitHub Models, Ollama, and OpenAI-compatible services
- More valuable by using **tools**, keeping **conversation state**, and collaborating through explicit **workflows**

SUMMARY

Quick Summary #2

- Clear **orchestration patterns** such as sequential, concurrent, handoff, and planner-led flows make agent behavior easier to explain, test, and evolve

- **Start simple** with one focused agent, then add memory, tools, and orchestration only when the use case requires it.

- Use workflows when **responsibilities need to be separated** across roles or when different specialists should contribute in a controlled way.

SOURCECODE

Find the code on GitHub

tsjdev-apps/maf-demo-console